

FLAT ASSIGNMENT

Name - Manan Shukla

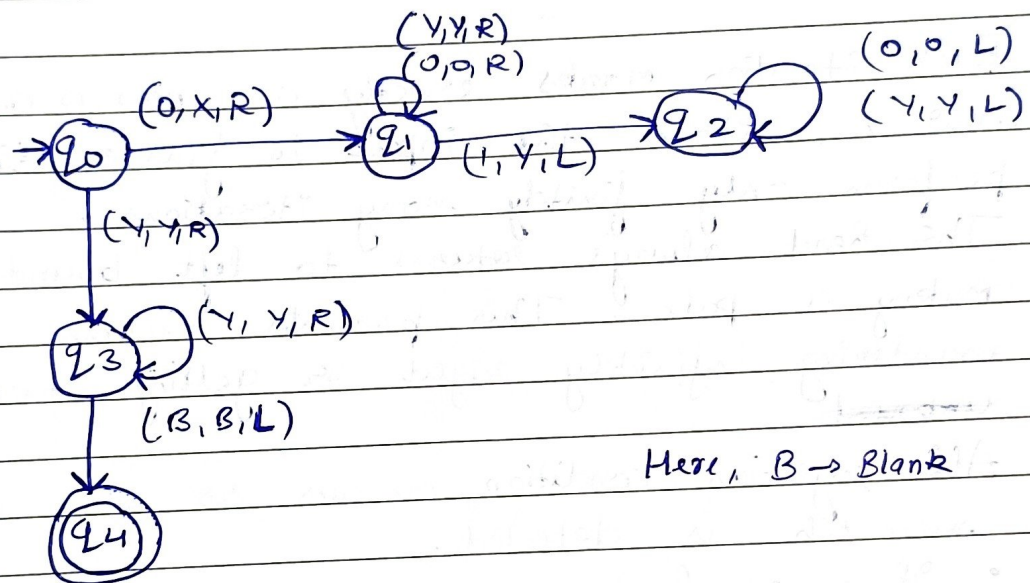
SAP ID - 560119574

Roll No - R2142230365

Batch - 1 (graphics & gaming)

Q1

Design a Turing machine that halts for all inputs
 $L = \{0^n 1^n \mid n \geq 1\}$



High Level Algo-

- a) Start at left end of tape
 Scan right until you find leftmost 'unmarked' 0
- b) Mark this 0 as X to indicate it has been processed
- c) Continue scanning right to find leftmost unmarked 1.
 - if such a 1 is found, mark it as Y
 - if no unmarked 1 appears before hitting blank symbol (B), reject. Because there is 0, without 1
- d) Return to left end of tape and begin next iteration.

e) ~~g~~

final check -

After all 0's have been marked X, verify that no unmarked 1 exists

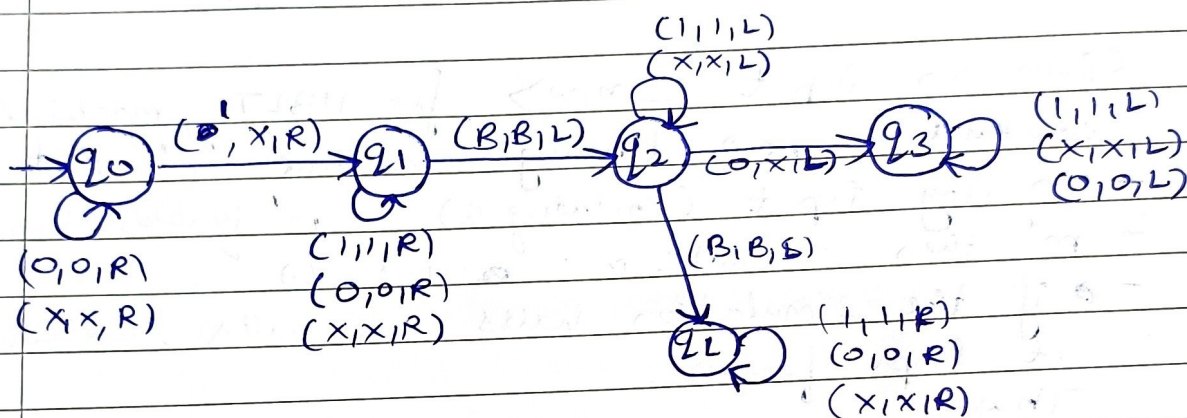
- If only X's and Y's remain, accept.
- If any unmarked 1's appear, reject.

Justification - The machine halts as -

- Each iteration marks exactly one new 0 and one new 1. Since the input is finite, the machine performs only finitely many iterations.
- The head always returns to left boundary after making a pair. This prevents machine from wandering infinitely right or getting stuck ~~in an unmarked~~.
- All rejection condition occurs as soon as mismatch is detected.
 - If a 0 lacks a corresponding 1, halt and reject.
 - If leftover 1 remains, halt and reject.
- When no unmarked 0 remains, machine performs one final linear scan and halts.
- No infinite loops are possible because in every iteration, machine marks new symbols or detects an invalid pattern and halts.

Q3 Construct ... certain points

$L \in \{w \in \{0,1\}^* \mid w \text{ contains more 1 than 0}\}$



The machine is a recognizer, so it must accept strings where $\#a > \#b$, but it is allowed to run forever on strings that don't satisfy condition. When machine searches for a matching pair of symbols and fails to find [when $\#a < \#b$], it doesn't reject. Instead it enters a non-halting looping state. Thus for any inputs like b, bb, ab etc where $\#a < \#b$ the machine never reaches accepting state and continues looping indefinitely.

Q4 Consider the ... proof sketch.

$HALT_L = \{ \langle M \rangle \mid M \text{ halts on empty string} \}$

~~This~~ This is undecidable...

Proof - Assume, for contradiction, that there exists a decider B for $HALT_L$

We will use R to build a decider S for Halting Problem

$\circ$ $HALT = \{ \langle m, w \rangle \mid m \text{ halts on } w \}$ which is known to be undecidable.

Given an input $\langle m, w \rangle$ for $HALT$, machine S constructs a new Turing machine ' m' ' that -

- on any input (including ϵ), ' m' ' ignores input
- ' m' ' then simulates ' m ' on input w
- if the simulation halts, ' m ' halts. Otherwise it loops forever.

Thus

$m' \text{ halts on } \epsilon \iff m \text{ halts on } w$

Now S runs on supposed decider on $\langle m' \rangle$.

- if R accepts, ' m ' halts on w
- if R rejects, ' m ' doesn't halt on w

This means S decides $HALT$, which is impossible. Therefore, our assumption that R exists is false.

Q5 Show that ----- procedure for it
To check whether a given Turing machine M belongs to this language, we can simulate m on all strings of length at most 3. Since this set of strings is finite, we can simulate all of them in parallel using a technique called forktail.

If M accepts any one of these strings, then simulation will eventually reach that accepting configuration [even it takes a very ~~small~~ large no. of steps, such as million or more). In this case, we can 'safely' conclude that $\langle m \rangle$ is in L .

However, main difficulty arises when M does not accept any of these strings. Some simulations may reject, while others may run forever. There is no algorithmic way to distinguish b/w a computation that'll never accept and the one that'll accept after an extremely large no. of steps.

Because of this limitation, we can never be completely sure that m will not accept one of strings in future. Therefore, although acceptance can be detected in finite time, non-acceptance can't be confirmed.

As a result, a Turing Machine can be constructed that accepts all machines in L , but it may run forever on machines that are not in L . Such a machine is called recognizer.

Hence, language ' L ' is recognizable but not decidable.

Q6 Prove your reduction
Language $EQ_{CFG} = \{ \langle G_1, G_2 \rangle \mid L(G_1) = L(G_2) \}$

Proof that it is undecidable

To prove EQ_{CFG} is undecidable, we reduce ALL_{CFG} to EQ_{CFG}

$ALL_{CFG} = \{ \langle G \rangle \mid L(G) = \Sigma^* \}$
it is undecidable

Given a CFG, we construct a new CFG G_0 such that

$$L(G_0) = \Sigma^*$$

Define $G_0 = (V, \Sigma, R, S)$ where

• $V = \{S\}$ (single start var.)

• for every terminal symbol $a \in \Sigma$, include the rule

$$S \rightarrow aS$$

$$S \rightarrow \epsilon$$

It is observed that this grammar generates all strings over Σ , so $L(G_0) = \Sigma^*$

Assume, for contradiction, that there is a Turing machine R that decides EQ_{CFG} . We construct a new machine S that decides ALL_{CFG} as follows-

On input $\langle G \rangle$:

1. Run R on pair $\langle G, G_0 \rangle$
2. If R accepts, accept.
3. If R rejects, reject.

- R checks whether

$$L(q) = L(q_0)$$

But since $L(q_0) = \Sigma^*$, this is equivalent to checking ~~not~~ whether

$$L(q) = \Sigma^*$$

So machine S decides ALL_{CFG}

But ALL_{CFG} is known to be undecidable. This is a contradiction, so our assumption about existence of R must be false

Since a decider for EQ_{CFG} would allow us to decide ALL_{CFG} and ALL_{CFG} is undecidable, it follows that

EQ_{CFG} is undecidable

Q7 State Cook to SAT.

- It states that SAT is NP complete
- It states that every problem in NP could be reduced in polynomial time to boolean satisfiability problem
- We can also say any non-deterministic polynomial time ~~deciding~~ Turing machine we can make Boolean formula whose assignment correspond exactly to accepting computation path of machine

Configuration of TM: 3 things

- Current state q
- Take const $\langle u, v \rangle$ symbol
- Head position v

Encoding Idea (1) Make Table

- make grid of size $T \times T$ representing row & col?
- Each has their own alphabet or pair (state & symbol)

(ii) Introduce boolean Var - For every time t , take ~~bottom~~ position L , symbols, we introduce new variable $X_{L,t} = 1$ iff time t , state s ,

(iii) Enclosing transaction rule - For 2×2 grid of upper row is having valid ~~operation~~ configuration then lower row must reflect applying TM transaction rules

(iv) Encode start configuration - Take initially contain input.

Head point at left most symbol.
State is q_0

(v) Encode acceptance condition - At least clause req. that some cells in last row contain accepting state.

Q8 Parameter $\rightarrow$ Input length $= w$
Run time $= f(n)$
Table size $= O(n^2)$, grid size $n \times w$

No. of variable $\rightarrow$ Boolean Var X_{ij} is defined for every cell (i,j) & symbols
grid size $= N \times N = O(n^2) \cdot O(n^2) = O(n^4)$
Symbol $= |F| + |A|$
Total variables $= N \times N \times C = O(n^2)^2$

- No. of clause -
- a) SAT formula. ϕ is composed of clause that checks validity condition for every cell or window of cell in grid.
- Cell consistency - Check each N^2 cell contains exactly one symbol.
It generates $O(n^2)$ clause
 - Transition - Check every cell of fellow in TM.
Since there is roughly one window for every cell in grid that generates $O(n^2)$ clause
 - Start & accept - Generate ϕ $O(n)$ & $O(N^2)$ clause
 - Total clause - Dominant term $O(N^2)$
Clause $O(n^2) = (O(n^2))^2 = O(n^4)$

Q9 $L = \{ \langle M \rangle \mid L(M) \text{ is finite} \}$

- a) Language of machine is finite
- b) This property of language $L(M)$ {Depend on set of string accepted not particular encoding M }
So it is a semantic property of Turing machine language

Why Rice Theorem applies? Every non trivial property for some recursively

enumerable language is undecidable.

- Non trivial means property hold for some recursively enumerable language & fail for some others.
- Our property is non trivial because there exists TM's whose lang is finite & exist TM whose lang. is finite.
This property is non trivial.

d10 Consider the ——— does not apply

The property stated here is,

"The machine accepts at least one string that contains subeking"

This depends only on language recognized by M , not on its structure. Therefore, this is a semantic property.

It is also non-trivial :

- some machines accept such strings
- some machines do not.

Since property is semantic & non trivial, Rice's theorem applies.

Thus, P is undecidable.